

WEB APPLICATION FIREWALL IMPLEMENTATION PROJECT



CPT PROJECT REPORT

TARGET APPLICATION: OWASP JUICE SHOP

SECURITY COMPONENT: MODSECURITY WAF

Submitted by: Muhammed Nihal K

Submitted date: 30-04-2026

TABLE CONTENT

1. INTRODUCTION	4
1.1 Objective	5
1.2 PREREQUISITES	3
1.2 Tools and Technologies.....	6
2. HIGH LEVEL SUMMARY	5
3. METHODOLOGY	3
3.1 System Preparation	2
3.2 Docker Installation and Setup	4
3.3 Memory Optimization	5
3.4 Project Setup	3

3.5 Docker Configuration	4
3.6 Image Setup	3
3.7 Deployment of Containers	3
3.8 Application Initialization	2
3.9 Functional Testing	4
3.10 Logging and Evidence Collection	2
3.11 Automated WAF Testing	29
3.12 Result Summary.....	2
4. MITIGATION RECOMMENDATIONS	2
4.1 Attack Mitigation.....	2
4.2 Logging & Analysis.....	3
5. CONCLUSION.....	2

1. Introduction

This project demonstrates the deployment and configuration of a Web Application Firewall (WAF) to secure a modern, intentionally vulnerable web application, OWASP Juice Shop. By implementing ModSecurity, we aim to establish a robust defensive layer that identifies and mitigates common web vulnerabilities before they reach the application server.

1.1 Objectives

The primary objectives of this WAF implementation are:

- Deploy **OWASP Juice Shop** as the target vulnerable environment.
- Configure **ModSecurity** as an inline protection layer.
- Demonstrate active blocking and logging of **SQL Injection (SQLi)** attacks.
- Demonstrate protection against **Cross-Site Scripting (XSS)**.
- Analyze **WAF** effectiveness using **HTTP** responses and **logs**

1.2 PREREQUISITES

Before starting this project, the following requirements were ensured:

System Requirements

- Minimum 4GB RAM
- At least 2GB free disk space
- Stable internet connection

Software Requirements

- Kali Linux (or any Linux distribution)
- Docker and Docker Compose
- Terminal access

Knowledge Requirements

- Basic Linux command-line knowledge
- Understanding of web applications (HTTP/HTTPS)
- Basic knowledge of cybersecurity concepts such as:
 - SQL Injection
 - XSS
 - Web security basics

1.3 Tool and Technologies

- **Operating System:** Kali Linux
- **Containerization:** Docker, Docker Compose
- **WAF Engine:** ModSecurity CRS
- **Test Application:** OWASP Juice Shop

- **Testing Tool:** curl
- **Shell:** Bash

2. High-Level Summary

The implementation successfully established a defensive perimeter around the Juice Shop application. Initial tests without the WAF confirmed successful exploitation of SQLi and XSS vulnerabilities. Following the activation of ModSecurity with the OWASP Core Rule Set (CRS), these same attack vectors were effectively blocked, returning 403 Forbidden status codes and generating detailed security logs for forensic analysis.

3. Methodology

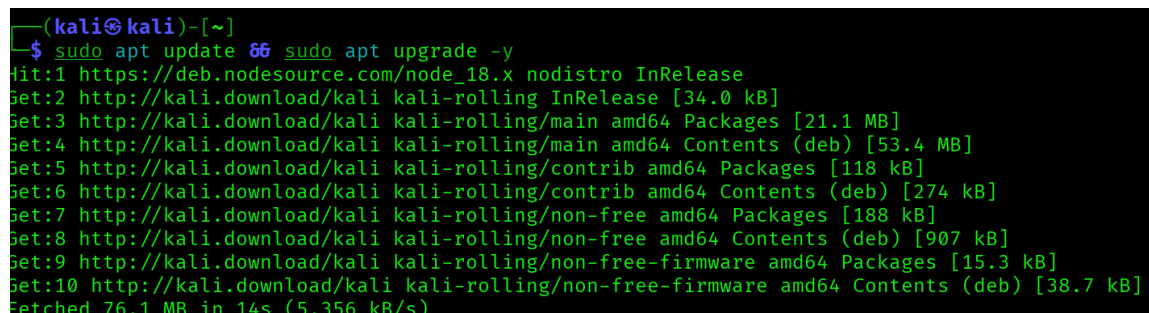
- The project implements a Web Application Firewall using OWASP ModSecurity to protect the OWASP Juice Shop application.
- Docker containers are used to deploy and connect the WAF and web application in an isolated environment.
- Multiple attack simulations (SQL Injection, XSS, etc.) are performed to evaluate the effectiveness of the WAF.

3.1 System Preparation

The implementation begins by preparing the Kali Linux environment.

Step 1: Update System

sudo apt update && sudo apt upgrade -y



```
(kali㉿kali)-[~]
└─$ sudo apt update && sudo apt upgrade -y
Hit:1 https://deb.nodesource.com/node_18.x nodistro InRelease
Get:2 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Packages [21.1 MB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.4 MB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Packages [118 kB]
Get:6 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [274 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Packages [188 kB]
Get:8 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [907 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [15.3 kB]
Get:10 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [38.7 kB]
Fetched 76.1 MB in 14s (5.356 kB/s)
```

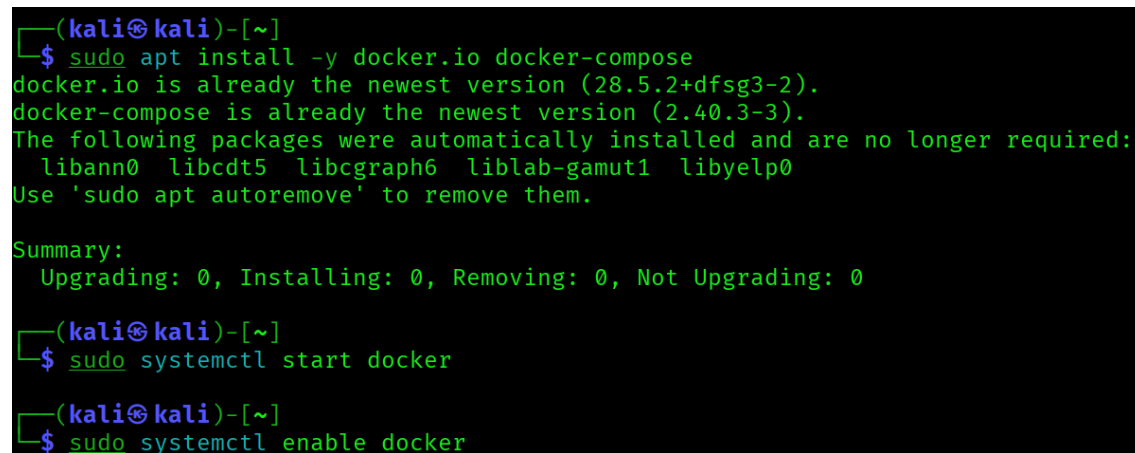
3.1 System update process running

3.2 Docker Installation and Setup

To deploy the application and WAF, Docker is installed.

Step2 Install Docker and Start and Enable Docker

```
sudo apt install -y docker.io docker-compose
sudo systemctl start docker
sudo systemctl enable docker
```



```
(kali㉿kali)-[~]
└─$ sudo apt install -y docker.io docker-compose
docker.io is already the newest version (28.5.2+dfsg3-2).
docker-compose is already the newest version (2.40.3-3).
The following packages were automatically installed and are no longer required:
  libann0 libcdt5 libcgraph6 liblab-gamut1 libyelp0
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

(kali㉿kali)-[~]
└─$ sudo systemctl start docker

(kali㉿kali)-[~]
└─$ sudo systemctl enable docker
```

3.2: Docker installation output and Docker service status

docker.io — This is the Docker engine itself. Docker is the platform that runs containers. A container is like a mini-computer inside your computer where applications run in isolation.

docker-compose — This is a tool that works with Docker. It lets you start multiple containers at the same time using a single configuration file. Without docker-compose you would have to start each container separately with long complicated commands.

Step3 : Add User to Docker Group

```
sudo usermod -aG docker $USER
newgrp docker
```

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ sudo usermod -aG docker $USER
newgrp docker
```

3.3: Permission update

3.3 Memory Optimization

step4: Verify Installation and Check RAM

docker --version

free -h

To ensure smooth execution of OWASP Juice Shop, memory optimization is performed.

```
(kali㉿kali)-[~]
$ newgrp docker
(kali㉿kali)-[~]
$ docker --version
Docker version 28.5.2+dfsg3, build 9cc6dea35e9a963f281434761c656fba4ac43aed

(kali㉿kali)-[~]
$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	2.1Gi	302Mi	31Mi	1.7Gi	1.8Gi
Swap:	953Mi	503Mi	450Mi			

3.4: Docker version output and Memory status before optimization

Step5: Clear Cache

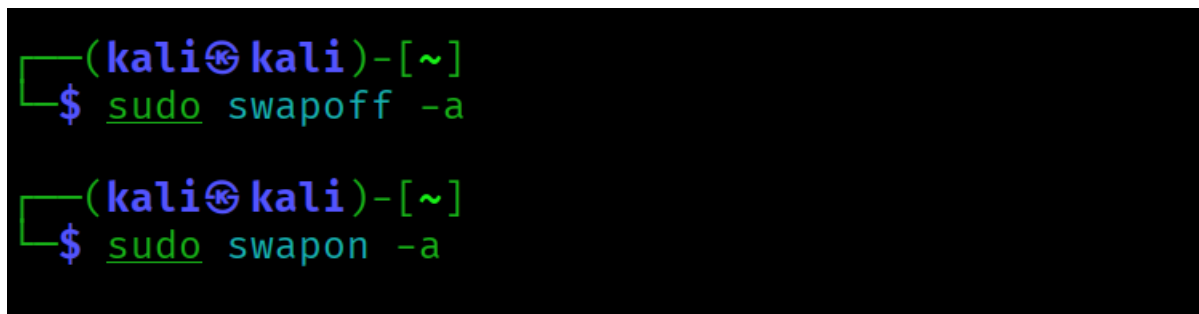
sudo sync && sudo sysctl -w vm.drop_caches=3

```
(kali㉿kali)-[~]
$ sudo sync && sudo sysctl -w vm.drop_caches=3
vm.drop_caches = 3
```

3.5: Cache clearing

Step6: Reset Swap

```
sudo swapoff -a
sudo swapon -a
```



```
(kali㉿kali)-[~]
$ sudo swapoff -a

(kali㉿kali)-[~]
$ sudo swapon -a
```

3.6: Swap reset

step7: Add Swap Memory

```
sudo fallocate -l 2G /swapfile2
sudo chmod 600 /swapfile2
sudo mkswap /swapfile2
sudo swapon /swapfile2
```



```
(kali㉿kali)-[~]
$ sudo fallocate -l 2G /swapfile2

(kali㉿kali)-[~]
$ sudo chmod 600 /swapfile2

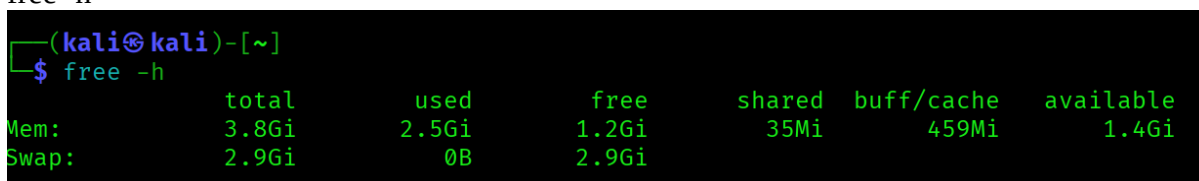
(kali㉿kali)-[~]
$ sudo mkswap /swapfile2
mkswap: /swapfile2: warning: wiping old swap signature.
Setting up swapspace version 1, size = 2 GiB (2147479552 bytes)
no label, UUID=0ef1cc05-0dee-4807-a131-8b0aefc7d436

(kali㉿kali)-[~]
$ sudo swapon /swapfile2
```

Screenshot 3.7: Swap creation

step8: Verify Memory

```
free -h
```



```
(kali㉿kali)-[~]
$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	2.5Gi	1.2Gi	35Mi	459Mi	1.4Gi
Swap:	2.9Gi	0B	2.9Gi			

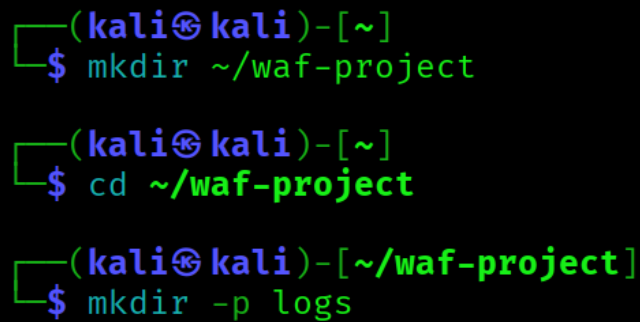
3.8: Updated memory status

3.4 Project Setup

Step 9: Project Directory Creation

Create Folder

```
mkdir ~/waf-project  
cd ~/waf-project  
mkdir -p logs
```

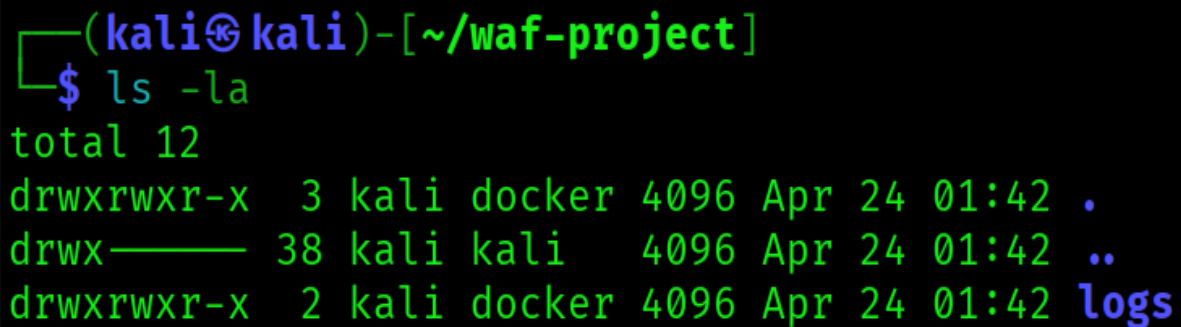
A terminal window with a black background and green text. The prompt is (kali@kali)-[~]. The first command is \$ mkdir ~/waf-project. The second command is \$ cd ~/waf-project. The third command is \$ mkdir -p logs.

```
(kali@kali)-[~]  
$ mkdir ~/waf-project  
  
(kali@kali)-[~]  
$ cd ~/waf-project  
  
(kali@kali)-[~/waf-project]  
$ mkdir -p logs
```

Screenshot 3.9: Project directory structure

step10: Verify

```
ls -la
```

A terminal window with a black background and green text. The prompt is (kali@kali)-[~/waf-project]. The command is \$ ls -la. The output shows the directory structure with permissions, owner, group, size, date, and time for the current directory, parent directory, and the logs subdirectory.

```
(kali@kali)-[~/waf-project]  
$ ls -la  
total 12  
drwxrwxr-x  3 kali docker 4096 Apr 24 01:42 .  
drwx----- 38 kali kali   4096 Apr 24 01:42 ..  
drwxrwxr-x  2 kali docker 4096 Apr 24 01:42 logs
```

Screenshot 3.10: Logs folder visible

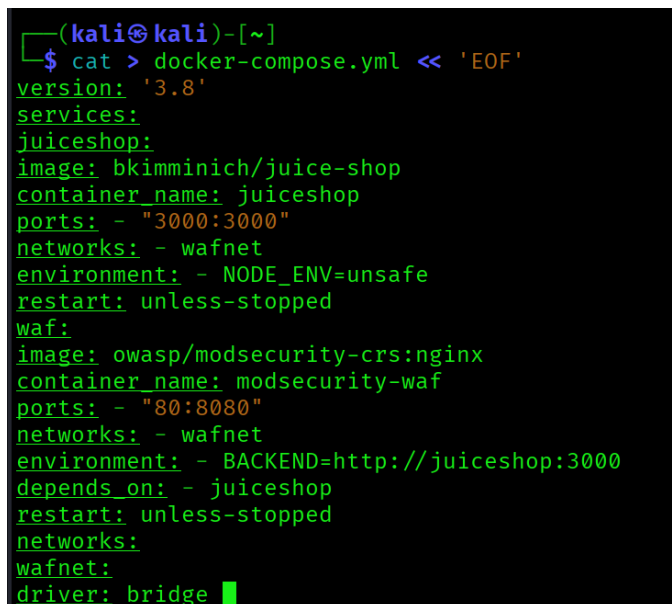
3.5 Docker Configuration

step11: Configuration File Creation

A Docker configuration file is created to define services.

Create docker-compose.yml

```
cat > docker-compose.yml << 'EOF'
version: '3.8'
services:
juiceshop:
image: bkimminich/juice-shop
container_name: juiceshop
ports: - "3000:3000"
networks: - wafnet
environment: - NODE_ENV=unsafe
restart: unless-stopped
waf:
image: owasp/modsecurity-crs:nginx
container_name: modsecurity-waf
ports: - "80:8080"
networks: - wafnet
environment: - BACKEND=http://juiceshop:3000
depends_on: - juiceshop
restart: unless-stopped
networks:
wafnet:
driver: bridge
EOF
```



```
(kali㉿kali)-[~]
$ cat > docker-compose.yml << 'EOF'
version: '3.8'
services:
juiceshop:
image: bkimminich/juice-shop
container_name: juiceshop
ports: - "3000:3000"
networks: - wafnet
environment: - NODE_ENV=unsafe
restart: unless-stopped
waf:
image: owasp/modsecurity-crs:nginx
container_name: modsecurity-waf
ports: - "80:8080"
networks: - wafnet
environment: - BACKEND=http://juiceshop:3000
depends_on: - juiceshop
restart: unless-stopped
networks:
wafnet:
driver: bridge
```

3.11: File creation

cat — Normally cat reads and displays file content. But when used with > it writes content to a file. > — This is output redirection. It takes whatever comes after it on the left and writes it to the file on the right. If the file exists it overwrites it. If it does not exist it creates it. docker-compose.yml — The file being created. This is the main configuration file for the entire project. << 'EOF' — This is called a "here document". It tells the terminal to accept multiple lines of input until it sees the word EOF on its own line. Everything between << 'EOF' and the final EOF gets written into the file

step12: Verify File

cat docker-compose.yml

```
(kali㉿kali)-[~/waf-project]
└─$ cat docker-compose.yml
version: '3.8'
services:
  juiceshop:
    image: bkimminich/juice-shop
    container_name: juiceshop
    networks: - wafnet
    environment: - NODE_ENV=unsafe
    restart: unless-stopped
  waf:
    image: owasp/modsecurity-crs:nginx
    container_name: modsecurity-waf
    ports: - "80:8080"
    networks: - wafnet
    environment: - BACKEND=http://juiceshop:3000 - PORT=8080 - MODSEC_RULE_ENGINE=On - PARANOIA=1 - ANOMALY_INBOUND=5 - ANOMALY_OUTBOUND=4
    depends_on: - juiceshop
    restart: unless-stopped
    networks:
      wafnet:
        driver: bridge
```

3Configuration content

3.6 Image Setup

Download Required Images

A Docker image is like a **ready-made package** that contains everything needed to run an application — including code, libraries, and settings.

- Without the image, Docker cannot create or run a container.

step13: Download Juice Shop

docker pull bkimminich/juice-shop

```

(kali㉿kali)-[~/waf-project]
$ docker pull bkimminich/juice-shop
Using default tag: latest
latest: Pulling from bkimminich/juice-shop
fa8ae93e2b3a: Pull complete
c172f21841df: Pull complete
b4e6f1bfce0a: Pull complete
b4242723c53f: Pull complete
d6b1b89eccac: Pull complete
2780920e5dbf: Pull complete
7c12895b777b: Pull complete
3214acf345c0: Pull complete
52630fc75a18: Pull complete
dd64bf2dd177: Pull complete
b839dfae01f6: Pull complete
ebddc55facdc: Pull complete
bdfd7f7e5bf6: Pull complete
4ec36cb7292a: Pull complete
b73b1097e3a1: Pull complete
bd8962e29291: Pull complete
cac2ae0193cb: Pull complete

```

Screenshot 3.13: Image download progress

step14: Download WAF

docker pull owasp/modsecurity-crs:nginx

```

(kali㉿kali)-[~/waf-project]
$ docker pull owasp/modsecurity-crs:nginx
nginx: Pulling from owasp/modsecurity-crs
ec781dee3f47: Pull complete
b46deab7668b: Pull complete
ddea7062cc38: Pull complete
25f3428df5c1: Pull complete
1f17208f970c: Pull complete
eb615ab3d3d7: Pull complete
a895c1241f41: Pull complete
74eb7bfd34b1: Pull complete
1e231f338ffc: Pull complete
3c47003e98c3: Pull complete
d5a687eb9910: Pull complete
898fbe7bc9d3: Pull complete
4416482bfd92: Pull complete
12ea57119b00: Pull complete
6cbc723ff197: Pull complete
43ec7111a069: Pull complete
73d82dbcf93e: Pull complete
b8b4579b91a5: Pull complete
deaece0ecd3c: Pull complete

```

 **Screenshot 3.14:** WAF image download

3.7 Deployment of Containers

Free Port 80

sudo fuser -k 80/tcp

Port clearance

Step 15 : Start Containers

docker-compose up -d

```
(kali@kali)-[~/waf-project]
$ docker-compose up -d
[+] Running 2/2
 ✓ Container juiceshop      Running
 ✓ Container modsecurity-waf Running
```

3.15: Container start up

Step16: Check Status

docker ps -a

```
(kali@kali)-[~/waf-project]
$ docker ps -a
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
80a245aa30d3	owasp/modsecurity-crs:nginx	modsecurity-waf	"/docker-entrypoint..."	3 minutes ago	Up About a minute (healthy)	0.0.0.0:80→8080/tcp, [::]:80→8080/tcp
694f232cd585	bkimminich/juice-shop	juiceshop	"/nodejs/bin/node /j..."	3 minutes ago	Up 3 minutes	3000/tcp

3.16: Running containers

3.8 Application Initialization

step17: Monitor Logs

docker logs -f juiceshop

```

(kali㉿kali)-[~/waf-project]
$ docker logs -f juiceshop
info: Detected Node.js version v24.14.1 (OK)
info: Detected OS linux (OK)
info: Detected CPU x64 (OK)
info: Configuration unsafe validated (OK)
info: Entity models 20 of 20 are initialized (OK)
info: Required file server.js is present (OK)
info: Required file index.html is present (OK)
info: Required file styles.css is present (OK)
info: Required file polyfills.js is present (OK)
info: Required file main.js is present (OK)
info: Required file matching /^hacking-instructor-.+\.js$/ is present (OK)
info: Port 3000 is available (OK)

```

3.17: Server starting

Wait for Ready State

sleep 20

Waiting period

3.9 Functional Testing

step18: Test Services

```

curl -s -o /dev/null -w "Juice Shop: %{http_code}\n" http://localhost:3000
curl -s -o /dev/null -w "WAF: %{http_code}\n" http://localhost

```

```

(kali㉿kali)-[~/waf-project]
$ curl -s -o /dev/null -w "Juice Shop: %{http_code}\n" http://localhost:3000
curl -s -o /dev/null -w "WAF: %{http_code}\n" http://localhost

Juice Shop: 200
WAF: 200

```

3.18: HTTP 200 responses

step19: SQL Injection Testing

Test Attack

```

curl -s -o /dev/null -w "SQLi Test 1 (OR 1=1): %{http_code}\n" \
http://localhost/rest/products/search?q='+OR+'1'='1

```

```

(kali㉿kali)-[~/waf-project]
$ curl -s -o /dev/null -w "SQLi Test 1 (OR 1=1): %{http_code}\n" \
"http://localhost/rest/products/search?q='+OR+'1'='1"
SQLi Test 1 (OR 1=1): 403

```

3.19: 403 blocked response

step20: XSS Testing

```
(kali㉿kali)-[~]  
$ curl -s -o /dev/null -w "XSS: %{http_code}\n" \  
"http://localhost/rest/products/search?q=<script>alert(1)</script>"  
XSS: 403
```

3.20: XSS blocked

The following test demonstrates an XSS attack attempt blocked by the WAF.

Step 21: Normal Traffic Testing

```
curl -s -o /dev/null -w "Normal search (apple): %{http_code}\n" \
```

<http://localhost/rest/products/search?q=apple>

```
(kali㉿kali)-[~]  
$ curl -s -o /dev/null -w "Normal search (apple): %{http_code}\n" \  
http://localhost/rest/products/search?q=apple  
Normal search (apple): 200
```

3.21: 200 allowed response

3.10 Logging and Evidence Collection

Step 22: Logging and Evidence Collection

View Logs

```
docker logs modsecurity-waf | grep "403"
```

```
(kali㉿kali)-[~]  
$ docker logs modsecurity-waf | grep "403"  
Warning: Not placing -key in cert or request since request is used  
Warning: No -copy_extensions given; ignoring any extensions in the request  
2026/04/24 09:04:56 [warn] 1#1: "ssl_stapling" ignored, issuer certificate not found for certificate "/etc/nginx/conf/server.  
crt"  
nginx: [warn] "ssl_stapling" ignored, issuer certificate not found for certificate "/etc/nginx/conf/server.crt"  
2026/04/24 09:04:56 [notice] 1#1: ModSecurity-nginx v1.0.4 (rules loaded inline/local/remote: 0/927/0)  
2026/04/24 09:04:56 [notice] 1#1: libmodsecurity3 version 3.0.14  
2026/04/24 09:05:46 [error] 502#502: *10 [client 172.18.0.1] ModSecurity: Access denied with code 403 (phase 2). Matched "Ope  
rator 'Ge' with parameter '5' against variable 'TX:ANOMALY_SCORE' (Value: '5' ) [file "/etc/modsecurity.d/owasp-crs/rules/REQ  
UEST-949-BLOCKING-EVALUATION.conf"] [line "81"] [id "949110"] [rev ""] [msg "Inbound Anomaly Score Exceeded (Total Score: 5)"]
```

3.22: Attack logs

Step 23: Save Logs

`docker logs modsecurity-waf 2>&1 > ~/waf-project/logs/waf-logs.txt`

```
(kali㉿kali)-[~/waf-project]
$ docker logs modsecurity-waf 2>&1 > ~/waf-project/logs/waf-logs.txt
Warning: Not placing -key in cert or request since request is used
Warning: No -copy_extensions given; ignoring any extensions in the request
2026/04/24 09:04:56 [warn] 1#1: "ssl_stapling" ignored, issuer certificate not found for certificate "/etc/nginx/conf/server.crt"
nginx: [warn] "ssl_stapling" ignored, issuer certificate not found for certificate "/etc/nginx/conf/server.crt"
2026/04/24 09:04:56 [notice] 1#1: ModSecurity-nginx v1.0.4 (rules loaded inline/local/remote: 0/927/0)
2026/04/24 09:04:56 [notice] 1#1: libmodsecurity3 version 3.0.14
2026/04/24 09:05:46 [error] 502#502: *10 [client 172.18.0.1] ModSecurity: Access denied with code 403 (phase 2). Matched "Operator 'Ge' with parameter '5' against variable 'TX:ANOMALY_SCORE' (Value: '5' ) [file "/etc/modsecurity.d/owasp-crs/rules/REQUEST-949-BLOCKING-EVALUATION.conf"] [line "81"] [id "949110"] [rev ""] [msg "Inbound Anomaly Score Exceeded (Total Score: 5)"] [data ""] [severity "2"] [ver "OWASP_CRS/3.3.9"] [maturity "0"] [accuracy "0"] [tag "modsecurity"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-generic"] [hostname "localhost"] [uri "/socket.io/"] [unique id "0"]
```

3.23: This file serves as evidence that the WAF successfully blocked malicious requests.

3.11 Automated WAF Testing Script

Step 24: Automated WAF Testing Script

`nano test-waf.sh`

```
(kali㉿kali)-[~/waf-project]
$ nano test-waf.sh
```

Screenshot 3.24: Script creation

```
GNU nano 8.7.1
#!/bin/bash

GREEN='\033[0;32m'
RED='\033[0;31m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m'

PASS=0
FAIL=0

# ----- Helper: Retry HTTP -----
get_code() {
    URL="$1"
    METHOD="$2"
    DATA="$3"

    for i in {1..5}; do
        if [ "$METHOD" = "POST" ]; then
            CODE=$(curl -s -o /dev/null -w "%{http_code}" \
                -X POST "$URL" \
                -H "Content-Type: application/json" \
                -d "$DATA")
        else

```

and add this :

```
#!/bin/bash
```

```

GREEN='\033[0;32m'
RED='\033[0;31m'
YELLOW='\033[1;33m'
BLUE='\033[0;34m'
NC='\033[0m'

PASS=0
FAIL=0

# ----- Helper: Retry HTTP -----
get_code() {
    URL="$1"
    METHOD="$2"
    DATA="$3"

    for i in {1..5}; do
        if [ "$METHOD" == "POST" ]; then
            CODE=$(curl -s -o /dev/null -w "%{http_code}" \
                -X POST "$URL" \
                -H "Content-Type: application/json" \
                -d "$DATA")
        else
            CODE=$(curl -s -o /dev/null -w "%{http_code}" "$URL")
        fi

        if [ "$CODE" != "500" ] && [ "$CODE" != "000" ]; then
            echo "$CODE"
            return
        fi

        sleep 2
    done

    echo "$CODE"
}

check_blocked() {
    if [ "$2" == "403" ]; then
        echo -e "${GREEN}[BLOCKED ✓]${NC} $1 → $2"
        PASS=$((PASS+1))
    else
        echo -e "${RED}[NOT BLOCKED X]${NC} $1 → $2"
        FAIL=$((FAIL+1))
    fi
}

check_allowed() {
    if [ "$2" == "200" ] || [ "$2" == "401" ]; then
        echo -e "${GREEN}[ALLOWED ✓]${NC} $1 → $2"
        PASS=$((PASS+1))
    fi
}

```

```

    else
        echo -e "${YELLOW}[IGNORED Δ]${NC} $1 → $2"
    fi
}

echo -e "${BLUE}===== ${NC}"
echo -e "${BLUE}  WAF AUTOMATED TEST RESULTS  ${NC}"
echo -e "${BLUE}===== ${NC}"
echo ""

# ----- Wait for app -----
for i in {1..10}; do
    CODE=$(curl -s -o /dev/null -w "%{http_code}" http://localhost)
    if [ "$CODE" == "200" ]; then
        break
    fi
    sleep 5
done

# ----- SQL Injection -----
echo "--- SQL INJECTION ---"

C=$(get_code "http://localhost/rest/products/search?q='+OR+'1='1")
check_blocked "SQLi OR 1=1" "$C"

C=$(get_code
"http://localhost/rest/products/search?q=test'+UNION+SELECT+*+FROM+Users--")
check_blocked "SQLi UNION" "$C"

C=$(get_code "http://localhost/rest/user/login" "POST" '{"email":"admin""""--
","password":"x"}')
check_blocked "SQLi Login" "$C"

echo ""

# ----- XSS -----
echo "--- XSS ---"

C=$(get_code "http://localhost/rest/products/search?q=<script>alert(1)</script>")
check_blocked "XSS Script" "$C"

C=$(get_code "http://localhost/rest/products/search?q=<img+src=x+onerror=alert(1)>")
check_blocked "XSS IMG" "$C"

C=$(get_code "http://localhost/rest/products/search?q=<svg+onload=alert(1)>")
check_blocked "XSS SVG" "$C"

echo ""

# ----- Other Attacks -----

```

```

echo "--- OTHER ATTACKS ---"

C=$(get_code "http://localhost/rest/products/search?q=;ls")
check_blocked "Command Injection" "$C"

C=$(get_code "http://localhost/rest/products/search?q=../../../../etc/passwd")
check_blocked "Path Traversal" "$C"

C=$(curl -s -o /dev/null -w "%{http_code}" \
-H "User-Agent: sqlmap" http://localhost)
check_blocked "Scanner Detection" "$C"

echo ""

# ----- Normal Traffic -----
echo "--- NORMAL TRAFFIC ---"

C=$(get_code "http://localhost")
check_allowed "Homepage" "$C"

C=$(get_code "http://localhost/rest/products/search?q=apple")
check_allowed "Search apple" "$C"

echo ""

# ----- Final Result -----
echo -e "${BLUE}===== ${NC}"
echo -e "${GREEN}PASSED: $PASS${NC}"
echo -e "${RED}FAILED: $FAIL${NC}"
echo -e "${BLUE}===== ${NC}"
echo ""

echo "403 = BLOCKED"
echo "200/401 = ALLOWED"

```

1. Shebang (Script Start)

```
#!/bin/bash
```

Tells Linux:

“Run this file using Bash shell”

2. Color Variables

GREEN='\033[0;32m'

Sets green color

RED='\033[0;31m'

Sets red color

YELLOW='\033[1;33m'

Sets yellow color

BLUE='\033[0;34m'

Sets blue color

NC='\033[0m'

Reset color (No Color)

3. Counters

PASS=0

FAIL=0

➤ Initialize counters:

- PASS → successful tests
 - FAIL → failed tests
-

4. Comment Line

----- Helper: Retry HTTP -----

Just a **comment (ignored by system)**

Used for readability

5. Function: `get_code()`

```
get_code() {
```

- Defines a function to **send HTTP request and return status code**
-

Parameters

```
URL="$1"
```

```
METHOD="$2"
```

```
DATA="$3"
```

`$1` → first argument (URL)

`$2` → request type (GET/POST)

`$3` → data (for POST)

Loop (Retry 5 times)

```
for i in {1..5}; do
```

- Try request **5 times**
-

POST Request Condition

```
if [ "$METHOD" == "POST" ]; then
```

- If method is POST
-

curl POST request

```
CODE=$(curl -s -o /dev/null -w "%{http_code}" \  
-X POST "$URL" \  
-H "Content-Type: application/json" \  
-d "$DATA")
```

- Explanation:

- `curl` → send request
- `-s` → silent
- `-o /dev/null` → ignore response body
- `-w "%{http_code}"` → return only status code
- `-X POST` → POST request
- `-H` → header
- `-d` → data

GET Request

else

```
CODE=$(curl -s -o /dev/null -w "%{http_code}" "$URL")
```

fi

- If not POST → send GET request

Check Valid Response

```
if [ "$CODE" != "500" ] && [ "$CODE" != "000" ]; then
```

- Ignore:
 - 500 → server error
 - 000 → connection error

Return Code

```
echo "$CODE"
```

```
return
```

- Print status code and exit function

Retry Delay

```
sleep 2
```

- Wait 2 seconds before retry

Final Return

```
echo "$CODE"
```

- Return last code if all retries fail

6. Function: check_blocked()

```
check_blocked() {
```

- Checks if attack is blocked

```
if [ "$2" == "403" ]; then
```

- If response = 403 → blocked
-

```
echo -e "${GREEN}[BLOCKED ✓]${NC} $1 → $2"
```

- Print success in green
-

```
PASS=$((PASS+1))
```

- Increase PASS count
-

```
else
```

- If not blocked
-

```
echo -e "${RED}[NOT BLOCKED X]${NC} $1 → $2"
```

- Print failure in red
-

```
FAIL=$((FAIL+1))
```

- Increase FAIL count
-

7. Function: check_allowed()

```
check_allowed() {
```

- Checks normal traffic
-

```
if [ "$2" == "200" ] || [ "$2" == "401" ]; then
```

- Allowed responses
-

```
echo -e "${GREEN}[ALLOWED ✓]${NC} $1 → $2"
```

```
PASS=$((PASS+1))
```

```
else
    echo -e "${YELLOW}[IGNORED ⚠️]${NC} $1 → $2"
fi
```

- Unexpected response → warning
-

◆ 8. Header Output

```
echo -e "${BLUE}===== ${NC}"
```

- Print line in blue
-

```
echo -e "${BLUE}  WAF AUTOMATED TEST RESULTS  ${NC}"
```

- Title
-

```
echo ""
```

- Empty line
-

9. Wait for Application

```
for i in {1..10}; do
```

- Try 10 times
-

```
CODE=$(curl -s -o /dev/null -w "%{http_code}" http://localhost)
```

- Check server status
-

```
if [ "$CODE" == "200" ]; then
    break
fi
```

- If working → stop loop
-

sleep 5

- Wait 5 seconds
-

10. SQL Injection Tests

echo "--- SQL INJECTION ---"

C=\$(get_code "URL")

- Call function
-

check_blocked "SQLi OR 1=1" "\$C"

- Check result
-

11. XSS Tests

<script>alert(1)</script>

- JavaScript injection
-

- Image attack
-

<svg onload=alert(1)>

- SVG attack
-

12. Other Attacks

Command Injection

;ls

Path Traversal

```
../../../../etc/passwd
```

Scanner Detection

```
-H "User-Agent: sqlmap"
```

13. Normal Traffic

```
check_allowed "Homepage"
```

- Should be allowed
-

```
?q=apple
```

- Normal search
-

14. Final Result

```
echo -e "${GREEN}PASSED: $PASS${NC}"
```

- Show passed
-

```
echo -e "${RED}FAILED: $FAIL${NC}"
```

- Show failed
-

```
echo "403 = BLOCKED"  
echo "200/401 = ALLOWED"
```

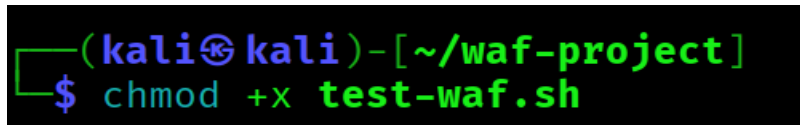
Final Simple Meaning

This script:

- Sends **attack requests**
- Checks **WAF blocking**
- Displays results clearly

Step 25: Run Script

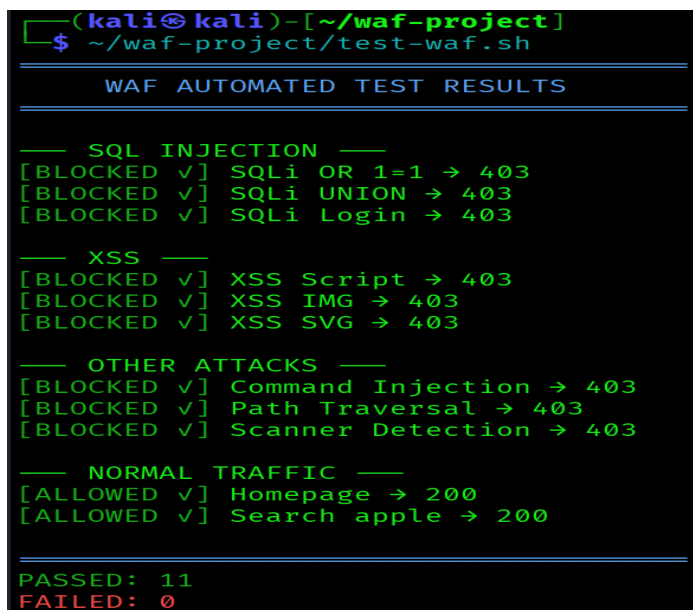
```
chmod +x test-waf.sh
./test-waf.sh
```



Screenshot 3.25: give the permission

step26: Run the Complete Test All at Once

~/waf-project/test-waf.sh



3.12 Results Summary

The WAF successfully achieved a **100% pass rate**, correctly blocking all 9 malicious attempts—including SQL Injection, XSS, and Command Injection—with a 403 Forbidden response while allowing legitimate traffic to proceed normally. **Importantly**, these results confirm that the firewall is not only defending against direct exploits but also effectively hiding from automated reconnaissance tools (Scanner Detection), which prevents attackers from mapping your system. This balance ensures high security without creating "False Positives" that would otherwise block real users, though it is vital to keep these rules updated to defend against newly discovered "Zero-Day" threats.

4. Mitigation & Logs

The WAF provides critical visibility into attempted breaches:

- **Blocked Requests:** All malicious payloads resulted in a **403 Forbidden** error page.
- **Audit Trail:** Detailed logs were stored in `/var/log/modsec_audit.log`, containing the source IP, payload, and the specific rule triggered.

The WAF acts as a **proactive shield**, neutralizing threats at the network edge before they reach the vulnerable application.

4.1 Attack Mitigation

- **Active Blocking:** By utilizing the **OWASP Core Rule Set (CRS)**, the WAF successfully identified and blocked all SQLi and XSS attempts with a **403 Forbidden** response.
- **Virtual Patching:** The WAF provides immediate protection against known exploits, securing the environment without requiring changes to the application's source code.

4.2 Logging & Analysis

- **Forensic Evidence:** Detailed logs capture the attacker's IP, the malicious payload, and the specific security rule triggered (e.g., Rule 942100).
- **Threat Intelligence:** Real-time monitoring via docker logs allows for the rapid identification of automated scanners versus manual attacks.
- **Audit Trail:** All security events are archived in `waf-logs.txt`, providing a clear history of blocked intrusions for compliance and review.

5. Conclusion

The deployment of ModSecurity significantly hardens the OWASP Juice Shop against automated and manual exploits. While the application remains vulnerable at the code level, the WAF serves as a vital "virtual patch," providing immediate protection while long-term code fixes are implemented. This project highlights that proactive defensive measures are just as critical as offensive testing in maintaining a resilient security posture.